

Examen: Poké-Quiz con JavaScript

Objetivo del Examen

En este examen práctico deberás desarrollar una **aplicación web interactiva de quiz temático de Pokémon**, conectando una interfaz frontend desarrollada con **HTML, CSS y JavaScript Vanilla** a un **servidor backend local en Node.js/Express**.

La aplicación muestra la **silueta de un Pokémon** y el alumno debe adivinar su nombre. Funcionalidades:

1. Consultar Pokémon desde el servidor backend local (`/api/pokemon?generacion= ... &random=true`) y las generaciones disponibles (`/api/generaciones`).
2. Mostrar la silueta del Pokémon en `#pokemonSprite` (aplicando la clase `.silueta` con filtro CSS negro) y al adivinar correctamente revelar la imagen real.
3. Renderizar el sprite del Pokémon, el feedback de acierto/error y el contenido de los modales de pista e historial en el DOM.
4. Implementar la lógica del quiz: validar la respuesta comparando con `toLowerCase()` / `trim()` , sumar puntos al `#score` , filtrar Pokémon por generación con `#generationSelector` y mostrar pistas desde el modal `#modalPista` .
5. Persistir el historial de partidas y el puntaje en `localStorage` y permitir su limpieza con `#btnLimpiarHistorial` .

Tabla de Entregas / Issues de GitHub

Cada entrega se corresponde con un **issue automático** en tu repositorio de GitHub. Para cerrar cada issue automáticamente, incluye el commit sugerido exacto al subir tu solución a la rama principal (`main`).

Entrega	Tarea a Realizar	Commit Sugerido
#1	Vincular <code>css/styles.css</code> y <code>js/script.js</code> en <code>index.html</code> .	<code>feat(html): vincular css y script js al html</code>
#2	Consumir la API local (<code>/api/pokemon?random=true</code>) usando <code>fetch</code> y <code>async/await</code> .	<code>feat(js): consumir api de pokemon con fetch y async await</code>
#3	Renderizar el sprite del Pokémon en <code>#pokemonSprite</code> , el feedback en <code>#feedback</code> y los modales en el DOM.	<code>feat(js): renderizar sprite del pokemon y modales en el dom</code>
#4	Implementar validación de respuesta, cálculo de puntaje, filtro por generación y apertura/cierre de modales.	<code>feat(js): implementar logica del quiz y filtrado por generacion</code>
#5	Persistir el historial de partidas en <code>localStorage</code> , mostrarlo en el modal y permitir su limpieza.	<code>feat(js): persistir y gestionar historial de partidas en localStorage</code>

🔧 Especificación Técnica y Requerimientos

1. Servidor Backend Local

El servidor Express provisto corre en el puerto `3000` con CORS habilitado:

- `GET http://localhost:3000/api/pokemon` : Devuelve la lista completa de Pokémon.
- `GET http://localhost:3000/api/pokemon?generacion=1` : Filtra Pokémon por generación.
- `GET http://localhost:3000/api/pokemon?random=true` : Devuelve un Pokémon aleatorio (objeto individual, no array).
- `GET http://localhost:3000/api/pokemon?generacion=1&random=true` : Pokémon aleatorio de una generación específica.
- `GET http://localhost:3000/api/pokemon/:id` : Devuelve un Pokémon por ID numérico o por nombre.
- `GET http://localhost:3000/api/generaciones` : Devuelve la lista de generaciones disponibles (`{ id, nombre }`).

Para iniciar el servidor backend:

```
npm start
```

2. Elementos Clave del DOM

- `#generationSelector` : `<select>` para filtrar Pokémon por generación (Gen I Kanto, Gen II Johto, etc.).
- `#score` : Elemento `<h2>` que muestra el puntaje acumulado.
- `#btnNuevo` : Botón para cargar un nuevo Pokémon aleatorio.
- `#btnPista` : Botón para abrir el modal de pista (`#modalPista`).
- `#btnHistorial` : Botón para abrir el modal de historial (`#modalHistorial`).
- `#pokemonSprite` : Imagen del Pokémon con clase `.silueta` (filtro CSS negro) que se revela al acertar.
- `#formQuiz` : Formulario con el input de respuesta y el botón de confirmación.
- `#guessInput` : Input de texto donde el alumno escribe el nombre del Pokémon.
- `#confirmarRespuesta` : Botón submit del formulario para validar la respuesta.
- `#feedback` : Párrafo donde se muestra si la respuesta fue correcta o incorrecta.
- `#modalPista` / `#hintContent` : Modal y contenedor del contenido de la pista (ej. tipo, generación, etc.).
- `#modalHistorial` / `#historyContainer` : Modal y contenedor donde se listan las partidas guardadas.
- `#btnLimpiarHistorial` : Botón para vaciar el historial en `localStorage` .
- `.cerrar-modal` : Botones para cerrar los modales (clase compartida).

3. Lógica del Quiz

- Al hacer click en `#btnNuevo` , se consulta `/api/pokemon?generacion=<gen>&random=true` y se muestra la silueta.
- El alumno escribe el nombre en `#guessInput` y envía el formulario `#formQuiz` .
- La validación compara la respuesta con `toLowerCase()` y `trim()` .
- Si acierta: se revela la imagen (remover clase `.silueta`), se suma al `#score` y se registra la partida.
- Si falla: se muestra un mensaje de error en `#feedback` .

4. Almacenamiento Local (`localStorage`)

- **Clave obligatoria:** `'pokedex_batallas'`
- **Estructura:** Arreglo de objetos con `{ pokemon, adivinado (boolean), score, fecha }`.
- Utilizar `JSON.stringify()` para guardar y `JSON.parse()` para leer.
- `#btnLimpiarHistorial` debe usar `removeItem()` o `clear()` para limpiar el historial.

Comandos de Prueba y Autoevaluación

Antes de entregar, podés autoevaluar tu trabajo localmente:

```
# Ejecutar todas las pruebas automáticas
npm test

# Ejecutar una prueba individual
npm run test:link
npm run test:fetch
npm run test:render
npm run test:events
npm run test:storage

# Validar estilo y calidad de código
npm run lint
npm run format:check
```

Instrucciones para la Ejecución Local

1. Instalar dependencias:

```
npm install
```

2. Iniciar el servidor local:

```
npm start
```

3. Abrir `index.html` en el navegador (usando la extensión **Live Server** de VS Code).
4. Abrir la consola de herramientas de desarrollador (**F12**) para verificar peticiones de red y depurar posibles errores.